

**A
PROJECT REPORT ON**

“Online Music Store”

**SUBMITTED IN
PARTIAL FULFILLMENT OF
DIPLOMA IN ADVANCED COMPUTING (PG-DAC)**



BY

**Dhairyashil Bhosale, Amit Patil,
Aditya Sudake, Saurabh Thorat**

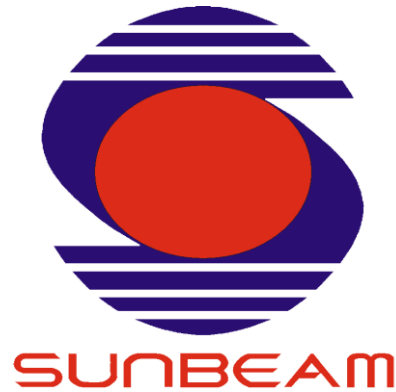
UNDER THE GUIDENCE OF

MR. RAHUL SANSUDDI

AT

**SUNBEAM INSTITUTE OF INFORMATION TECHNOLOGY,
PUNE**

**SUNBEAM INSTITUTE OF INFORMATION TECHNOLOGY,
PUNE.**



CERTIFICATE

This is to certify that the project

“Online Music Store”

Has been submitted by

**Dhairyashil Bhosale, Amit Patil,
Aditya Sudake, Saurabh Thorat**

In partial fulfillment of the requirement for the Course of **PG Diploma in Advanced Computing (PG-DAC AUG2015)** as prescribed by The **CDAC ACTS, PUNE.**

Place: Pune

Date: 30-JAN-2016

**Mr. Rahul Sansuddi
Project Guide**

**<Alumni Mentor Name>
Alumni Mentor**

ACKNOWLEDGEMENT

A project usually falls short of its expectation unless aided and guided by the right persons at the right time. We avail this opportunity to express our deep sense of gratitude towards Mr. Sarang Patil (Director, SIIT Karad), Mr. Prashant Lad (Course Coordinator, SIIT, Karad) and Mr. Rahul Sansuddi (Our Project Guide and Senior Faculty Member, SIIT Karad). We are deeply indebted and grateful to them for their guidance, encouragement and deep concern for our project. Without their critical evaluation and suggestions at every stage of the project, this project could never have reached its present form. Last but not the least we thank the entire faculty, especially Mr. Nitin Jadhav, and the staff members of Sunbeam Institute of Information Technology, Karad for their support.

TEAM SUNBEAM
(DAC-15 to DAC-27)
DAC August 2025 Batch,
SIIT Karad

ABSTRACT

INDEX

1. INTRODUCTION	1
1.1 Introduction	2
2. PRODUCT OVERVIEW AND SUMMARY	
2.1 Purpose	
2.2 Scope	
2.3 User Classes and Characteristics	
2.4 Design and Implementation Constraints	
3. REQUIREMENTS	
3.1 Functional Requirements	
3.1.1 Use case for Administrator.	
3.1.2 Use case for Customer.	
3.2 Non - Functional Requirements	
3.2.1 Usability Requirement	
3.2.2 Performance Requirement	
3.2.3 Reliability Requirement	
3.2.4 Portability Requirement	
3.2.5 Security Techniques	
4. PROJECT DESIGN	
4.1 Data Model	
4.1.1 Database Design	
4.2 Process Model	
4.2.1 Functional Decomposition Diagram	
4.2.2 Data Flow Diagram (DFD)	
5. TEST REPORT	
6. PROJECT RELATED STATISTICS	
7. CONCLUSION	

LIST OF TABLES

[illegible]

LIST OF FIGURES

[illegible]

1. Introduction

This project is all about developing a dynamic web-based subsystem, which facilitates the uploading, management, streaming, and commercial distribution of digital music through a secure, role-based environment.

The first part of the project is for establishing the application architecture and administrative control. This has been implemented by using React + Vite for a responsive frontend interface and Spring Boot for a robust backend API. The actors for this part are the Admin and the Artist. The Admin is responsible and authorized to oversee the entire platform ecosystem; this includes managing the user base (both Artists and Listeners), suspending users for policy violations, and moderating content by deleting songs if necessary. The Artist is the primary content contributor, authorized to upload multimedia assets—specifically MP3 files and cover images—and edit song metadata to maintain the music library.

The second part of the project focuses on the consumption and monetization of the content. The primary actor in this part is the Listener. The system allows Listeners to browse and stream songs online seamlessly. To differentiate service levels, the Admin has the capability to assign Premium status to specific Listeners.

The final part of the project handles the integration of secure financial transactions and exclusive feature access. This includes the implementation of the Cashfree payment gateway, which facilitates the upgrade process for Listeners. Once a Listener secures a premium subscription, the system unlocks additional privileges, specifically the ability to download songs for offline playback, thereby enhancing the standard streaming experience.

2. Project Overview and Summary

2.1 Purpose:

The primary purpose of the "**Online Music Store**" is to design and develop a scalable, web-based platform that bridges the gap between music creators and listeners through a centralized digital marketplace. This project aims to facilitate the seamless distribution, management, and consumption of digital music by leveraging a modern full-stack architecture using **React + Vite** for the frontend and **Spring Boot** for the backend.

The specific objectives of this system are as follows:

- **To Implement Secure Role-Based Access Control (RBAC):** To define clear operational boundaries and security protocols by distinguishing between Administrators, Artists, and Listeners. This ensures that administrative tasks, such as user suspension and content moderation, are restricted to authorized personnel.
- **To Empower Content Creators (Artists):** To provide a dedicated interface for Artists to upload, manage, and edit their musical content (MP3 files and cover images), allowing them to maintain an up-to-date portfolio without external intervention.
- **To Enhance User Experience and Accessibility:** To offer Listeners a responsive environment for browsing and streaming music online, ensuring high availability and low latency.
- **To Facilitate Monetization and Premium Services:** To integrate a secure financial transaction system using the **Cashfree payment gateway**. This purpose extends to differentiating service levels, where premium subscribers are granted exclusive rights to download content for offline usage, thereby adding commercial value to the platform.

2.2. Scope:

The scope of the "**Online Music Store**" is defined by the functionalities available to its three distinct user roles (Admin, Artist, Listener) and the technical boundaries of the web-based system. The project encompasses the following key areas:

1. Administrative Control (Admin Module)

The scope includes full control over the platform's ecosystem to ensure compliance and smooth operation.

- **User Management:** The system facilitates the administration of all user accounts, including the ability to suspend or reactivate users (Artists and Listeners) based on platform policies.

- **Content Moderation:** Administrators can remove content (songs) that violates terms of service or copyright regulations.
- **Subscription Management:** The scope covers the ability to designate specific Listeners as "Premium" users, thereby unlocking advanced features.

2. Content Management (Artist Module)

The project provides a dedicated workspace for content creators to manage their intellectual property.

- **Media Uploads:** The system supports the uploading of complex media formats, specifically MP3 audio files and associated image files (album art).
- **Metadata Editing:** Artists can edit song details to ensure accurate categorization and searchability within the store.

3. User Interaction & Consumption (Listener Module)

The primary interface for end-users focuses on accessibility and content delivery.

- **Online Streaming:** The system allows valid users to stream audio content in real-time over the web.
- **Premium Features:** The scope includes a downloadable content feature restricted strictly to Premium users.
- **Payment Integration:** Integration with the **Cashfree payment gateway** is included to handle secure transactions for upgrading Listeners to Premium status.

4. Technical Boundaries

- **Frontend:** The scope is limited to a web-based interface developed using **React and Vite**, ensuring compatibility with modern web browsers.
- **Backend:** The logic and data processing are confined to a **Spring Boot** architecture.
- **Security:** The system scope covers Role-Based Authentication (RBAC) to strictly enforce the separation of duties between the three user types.

2.3. User Classes and Characteristics:

The "**Online Music Store**" system identifies three primary categories of users (User Classes). Each class has distinct permissions, characteristics, and interactions with the system, enforced through Role-Based Access Control (RBAC).

1. Administrator (ADMIN)

- **Characteristics:** The Administrator is the super-user of the system with the highest level of privileges. This user class is responsible for the overall maintenance, moderation, and business logic of the platform. They act as the gatekeepers of the system's integrity.

2. Artist (ARTIST)

- **Characteristics:** The Artist is the primary content provider for the platform. This user class represents musicians or creators who utilize the system to distribute their intellectual property to a wider audience. Their interaction is focused on the *creation* and *maintenance* of the music library.

3. Listener (LISTENER)

- **Characteristics:** The Listener is the end-user or consumer of the application. This class is divided into two operational states: **Standard Listener** and **Premium Listener**. Their primary characteristic is content consumption rather than creation.

2.4. Design and Implementation Constraints:

Online Music Store are governed by several technical, operational, and security-related constraints. These constraints define the boundaries within which the system must function effectively.

- **Technology Stack Constraint:**
The system is strictly implemented using React with Vite for the frontend and Spring Boot for the backend. All design decisions, integrations, and optimizations are constrained to the capabilities and limitations of these technologies.
- **Web-Based Deployment Constraint:**
The application is developed exclusively as a web-based platform. It must operate within the limitations of modern web browsers and does not provide native mobile or desktop application support.
- **Security and Access Control Constraint:**
The system enforces Role-Based Access Control (RBAC). Each user action is restricted according to assigned roles (Admin, Artist, Listener), and unauthorized access to sensitive operations **such as** content moderation, user suspension, and premium feature access is prohibited.
- **Media Storage and Streaming Constraint:**
The platform supports only predefined media formats, primarily MP3 audio files and image files for cover art. Streaming performance is dependent on server bandwidth and network stability.
- **Payment Gateway Dependency Constraint:**
Premium subscription functionality relies on the Cashfree payment gateway. Any

downtime, latency, or API changes from the third-party service directly affect payment processing and subscription upgrades.

- **Performance and Scalability Constraint:**

System performance is influenced by concurrent user load, server resources, and database optimization. Large-scale traffic may require additional infrastructure scaling beyond the current project scope.

- **Legal and Copyright Compliance Constraint:**

The platform assumes that Artists upload content for which they have legal rights. The system does not implement automated copyright verification mechanisms, relying instead on administrative moderation.

- **Time and Resource Constraint:**

As an academic CDAC project, development is constrained by limited time, team size, and available computational resources, which restrict the implementation of advanced features such as AI-based recommendations or offline caching.

3. REQUIREMENTS

3.1 Functional Requirements

Functional requirements define the core operations and services provided by the system for each user role.

3.1.1 Use Case for Administrator (ADMIN)

The Administrator is responsible for managing the overall platform operations and enforcing system policies.

- The system shall allow the Administrator to log in securely using authenticated credentials.
- The system shall allow the Administrator to view, manage, suspend, or reactivate user accounts (Artists and Listeners).
- The system shall allow the Administrator to approve or remove uploaded songs that violate platform policies or copyright rules.
- The system shall allow the Administrator to monitor system activity and maintain platform integrity.
- The system shall restrict all administrative operations strictly to users with the ADMIN role.

3.1.2 Use Case for Customer (Listener)

The Customer (Listener) is the end-user who consumes music content through the platform.

- The system shall allow the Listener to register and log in securely.
- The system shall allow the Listener to browse the music catalog by song, artist, or category.
- The system shall allow the Listener to stream music online.
- The system shall allow the Listener to upgrade to Premium status through secure online payment.
- The system shall allow Premium Listeners to download songs for offline playback.
- The system shall restrict download functionality for non-premium users.
- The system shall allow the Listener to log out securely from the system.

3.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the system.

3.2.1 Usability Requirement

- The system shall provide a simple, intuitive, and responsive user interface.
- The application shall be easy to navigate for users with minimal technical knowledge.
- The system shall provide clear error messages and user feedback for all actions.
- The interface shall be consistent across all supported browsers.

3.2.2 Performance Requirement

- The system shall support multiple concurrent users without performance degradation.
- The system shall ensure minimal buffering during online music streaming under normal network conditions.
- The system shall process user requests and payments within an acceptable response time.
- The backend APIs shall be optimized for efficient data retrieval and media delivery.

3.2.3 Reliability Requirement

- The system shall ensure high availability for music streaming and user access.
- The system shall handle unexpected failures gracefully without data loss.
- The system shall maintain data consistency for user accounts, subscriptions, and media content.
- The system shall ensure successful transaction confirmation before upgrading Premium access.

3.2.4 Portability Requirement

- The system shall be accessible on all modern web browsers such as Chrome, Firefox, and Edge.
- The application shall be platform-independent and usable on different operating systems.

- The system shall not require any client-side installation beyond a standard web browser.

3.2.5 Security Techniques

- The system shall implement **Role-Based Access Control (RBAC)** to restrict functionality based on user roles.
- The system shall enforce secure authentication and authorization mechanisms.
- Sensitive data such as passwords shall be stored using encryption or hashing techniques.
- The system shall use HTTPS to ensure secure data transmission.
- The system shall integrate securely with the Cashfree payment gateway for financial transactions.
- The system shall prevent unauthorized access to premium content and administrative operations.

4. PROJECT DESIGN

4.1 Data Model

The data model defines how information is structured, stored, and managed within the **SCOTIFY** system. It ensures efficient data organization, integrity, and retrieval across all modules. The model supports role-based operations and maintains relationships between users, songs, subscriptions, and transactions.

4.1.1 Database Design

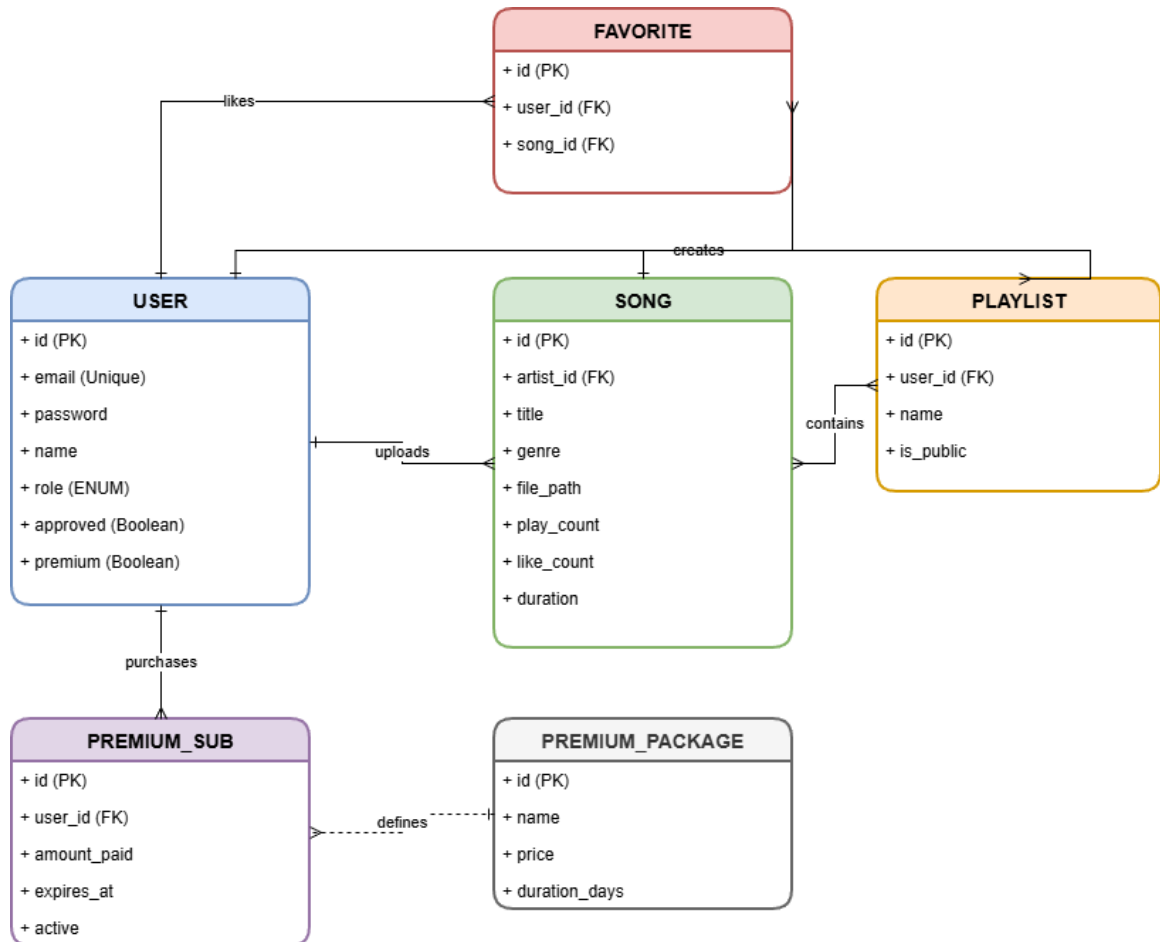
The database is designed using a relational model to support scalability and consistency. The core entities and their relationships are as follows:

- **User Table**
Stores user details including user ID, name, email, password, role (ADMIN, ARTIST, LISTENER), account status, and timestamps.
- **Artist Profile Table**
Maintains artist-specific information such as biography and uploaded content linkage.
- **Song Table**
Stores song metadata including song ID, title, genre, duration, upload date, file path, and artist ID.
- **Album / Cover Image Table**
Stores image metadata related to songs or albums.
- **Subscription Table**
Maintains subscription details such as listener ID, subscription type, start date, and expiry date.
- **Payment Table**
Stores transaction details including payment ID, listener ID, amount paid, payment status, and transaction timestamp.

Relationships:

- One Artist can upload multiple Songs.
- One Listener can have one active Subscription.
- One Subscription can be associated with multiple Payments.
- Admin oversees all entities but does not own records.

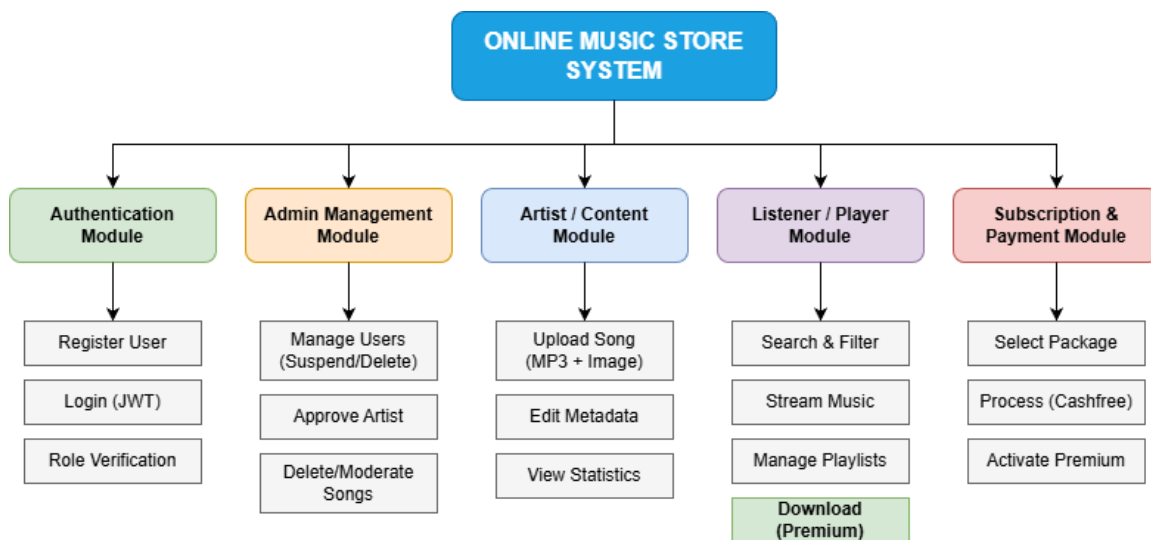
The database design ensures normalization, minimizes redundancy, and enforces referential integrity using primary and foreign keys.



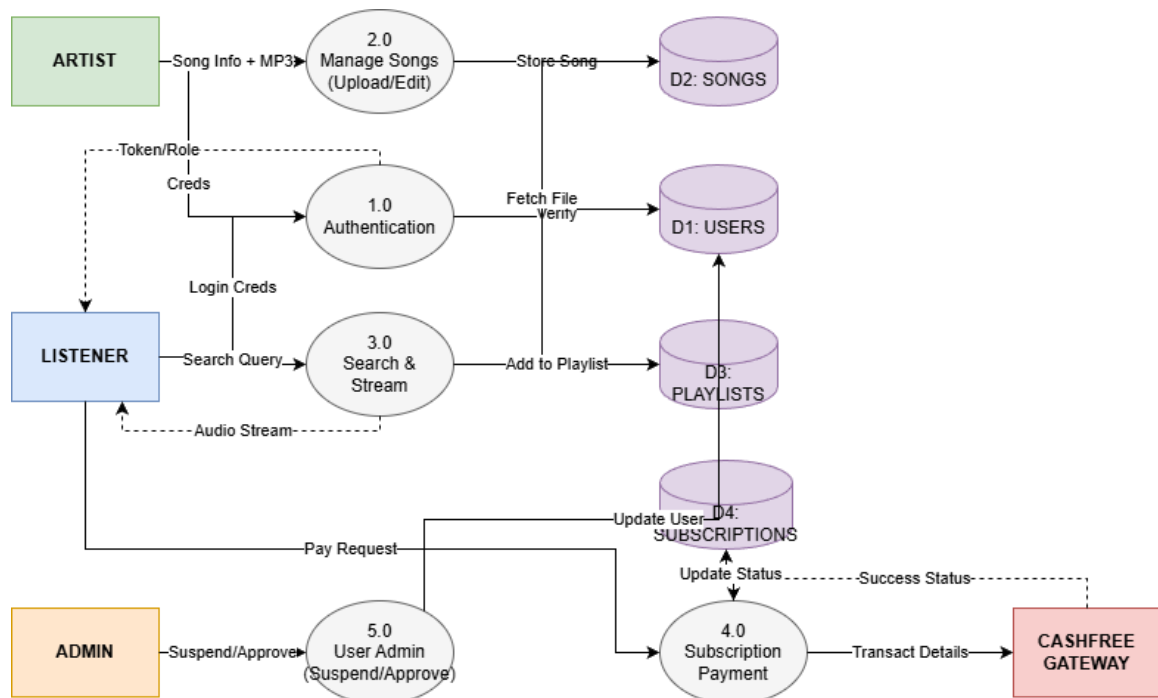
4.2 Process Model

The process model describes the logical flow of operations and interactions between system components and users. It illustrates how data moves through the system and how each functional module collaborates to achieve business objectives.

4.2.1 Functional Decomposition Diagram



4.2.2 Data Flow Diagram (DFD):



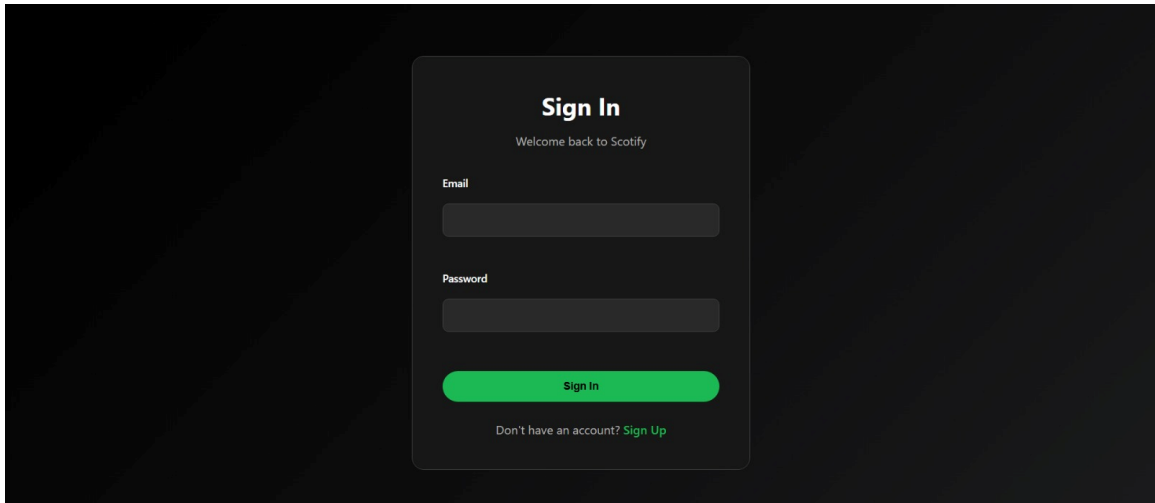
5. TEST REPORT

Sr. No.	Test Case Description	Test Steps	Expected Result	Actual Result	Error Message / Remark	Status
01	Verify User Registration	1. Enter valid Name, Email, Password. 2. Select Role (Listener). 3. Click Register.	User registered successfully.	User registered and token generated.	N/A	PASS
02	Verify Login with Invalid Password	1. Enter valid Email. 2. Enter wrong Password. 3. Click Login.	System should block login.	Login failed as expected.	"Bad credentials"	PASS
03	Verify Upload by Unapproved Artist	1. Log in as Unapproved Artist. 2. Try to upload a song.	System should block the upload.	Upload blocked (403 Forbidden).	"Your artist account is not approved for uploading content"	PASS
04	Verify Upload by Approved Artist	1. Log in as Approved Artist. 2. Upload valid MP3 and details.	Song should be saved successfully.	Song uploaded with ID generated.	N/A	PASS
05	Verify Suspend User	1. Admin sends suspend	User status updates to enabled=false.	User suspended successfully.	N/A	PASS

Sr. No.	Test Case Description	Test Steps	Expected Result	Actual Result	Error Message / Remark	Status
		request for a user. 2. Reason: "Policy Violation".				
06	Verify Suspended User Login	1. Suspended User attempts to log in.	System should block access.	Login failed.	"User is disabled"	PASS
07	Verify Song Streaming	1. Listener clicks Play on a song.	Song plays without error.	Audio streams (Status 206).	N/A	PASS
08	Verify Download for Free User	1. Free Listener clicks "Download".	System should restrict access.	Access denied (403 Forbidden).	"Premium subscription required to download songs"	PASS
09	Verify Download for Premium User	1. Premium Listener clicks "Download".	File download should start.	File downloaded successfully.	N/A	PASS
10	Verify Payment Processing	1. User completes payment via Cashfree.	Subscription updates to active.	Premium active until expiry.	N/A	PASS
11	Verify Add Song to Playlist	1. User adds "Song A" to "Playlist B".	Song is linked to playlist.	Song added successfully.	N/A	PASS
12	Verify Access Non-Existent Playlist	1. Request playlist with Invalid ID (e.g., 9999).	System should return error.	Request failed (404 Not Found).	"Playlist not found"	PASS

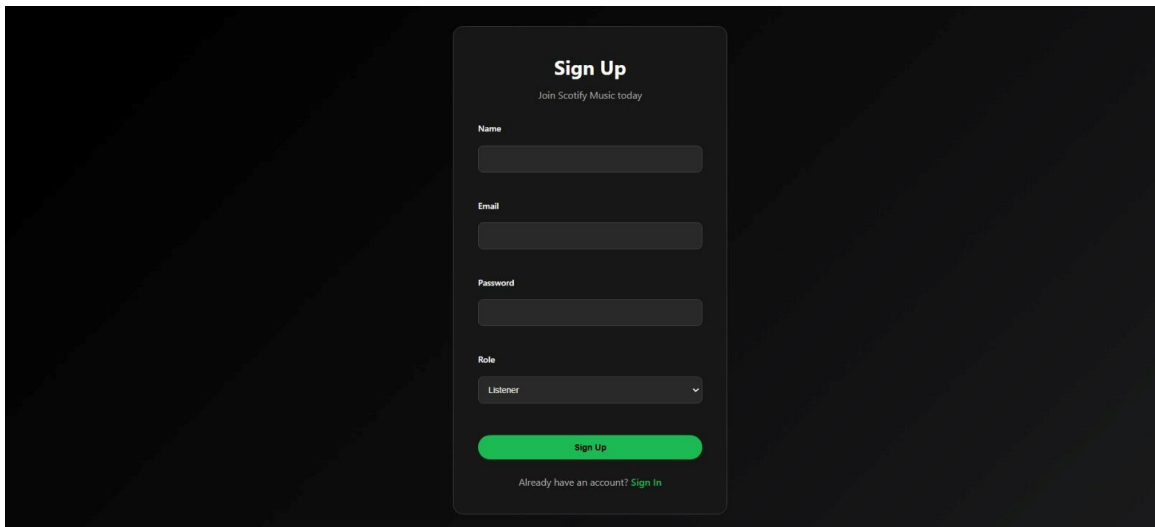
6. PROJECT SCREENSHOTS

6.1. Login Page:



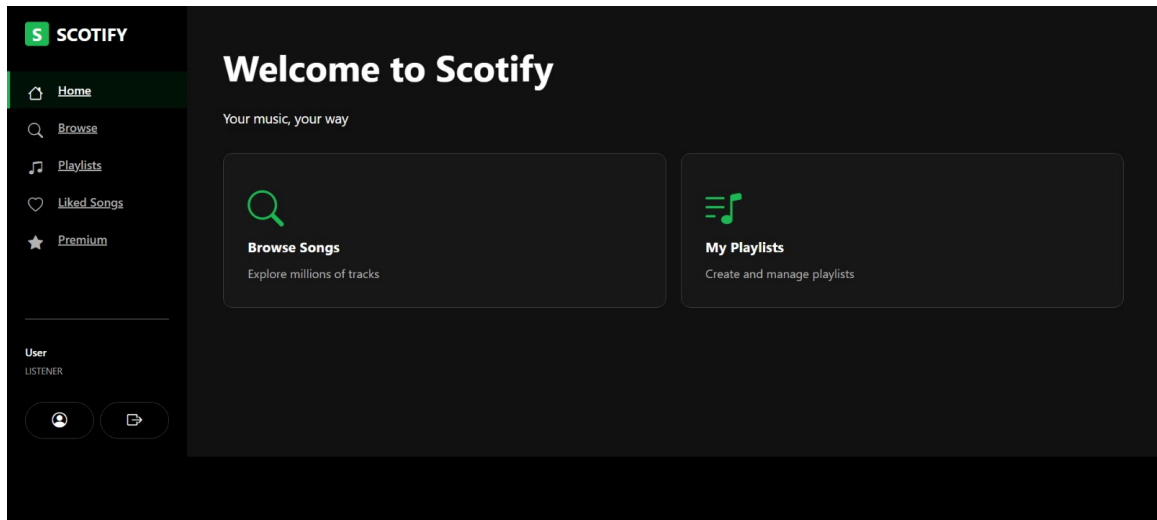
The screenshot shows a dark-themed login page. A central light gray rounded rectangle contains the 'Sign In' form. At the top of the form is the title 'Sign In' in bold, followed by the subtitle 'Welcome back to Scotify'. Below these are two input fields: 'Email' and 'Password'. A bright green 'Sign In' button is positioned below the password field. At the bottom of the form, there is a link that says 'Don't have an account? Sign Up'.

6.2. Sign-Up Page:

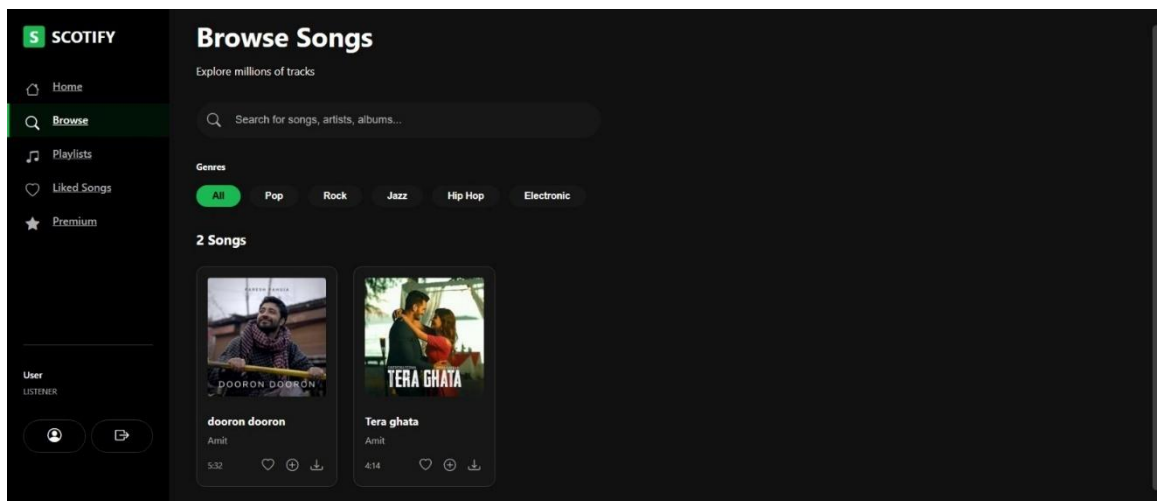


The screenshot shows a dark-themed sign-up page. A central light gray rounded rectangle contains the 'Sign Up' form. At the top of the form is the title 'Sign Up' in bold, followed by the subtitle 'Join Scotify Music today'. Below these are four input fields: 'Name', 'Email', 'Password', and 'Role'. The 'Role' field is a dropdown menu with 'Listener' selected. A bright green 'Sign Up' button is positioned below the role field. At the bottom of the form, there is a link that says 'Already have an account? Sign In'.

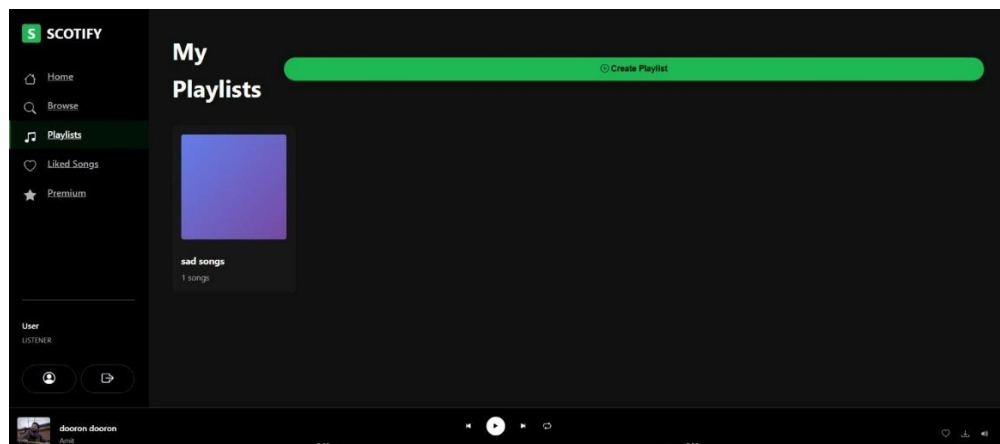
6.3. Listener Dashboard:



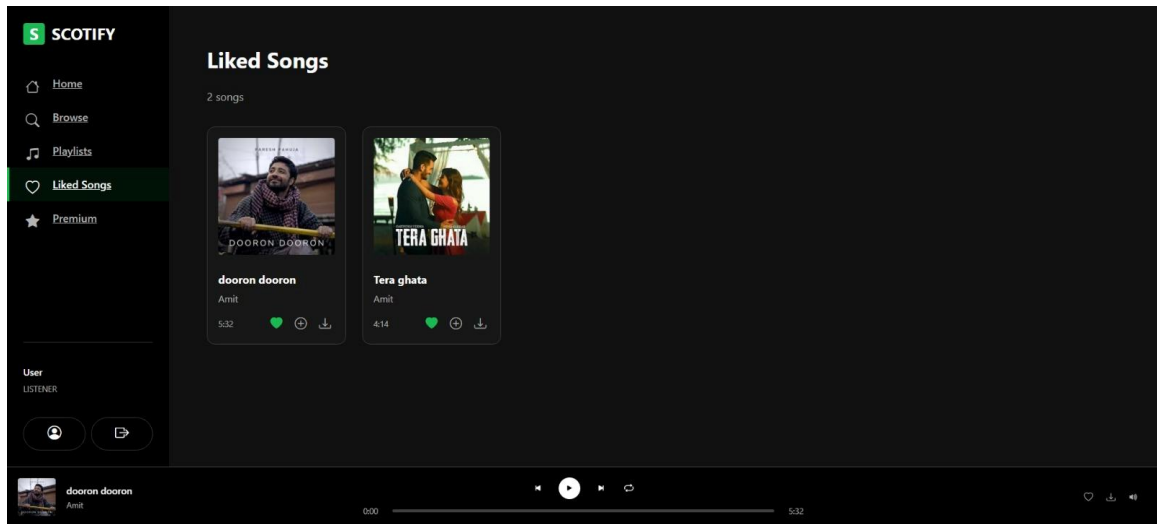
6.4. Listener Browse Page:



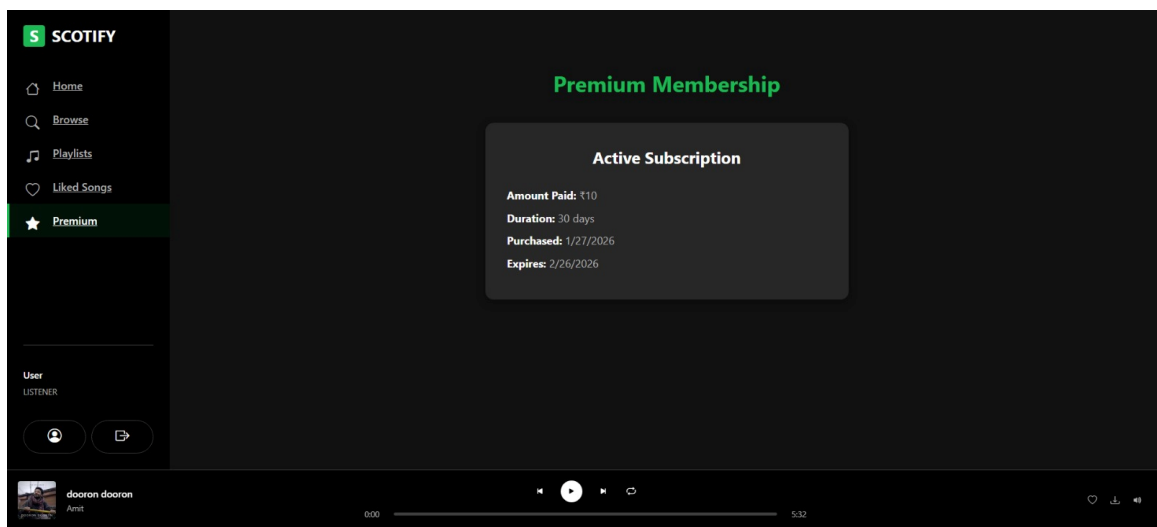
6.5. Playlist Page:



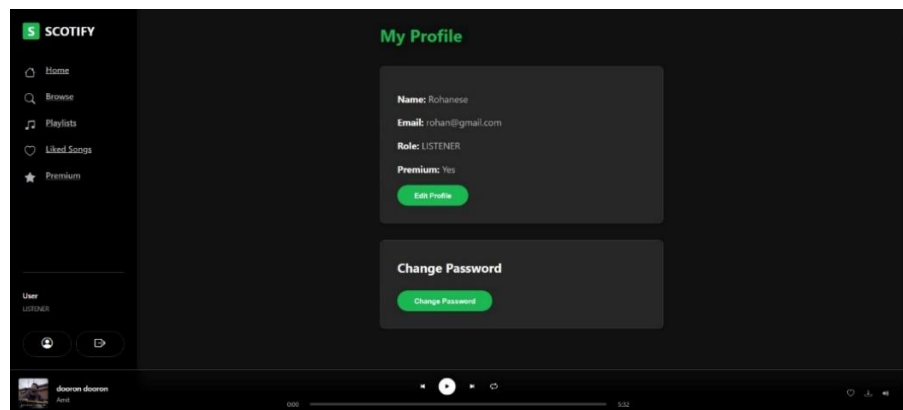
6.6. Liked Songs:



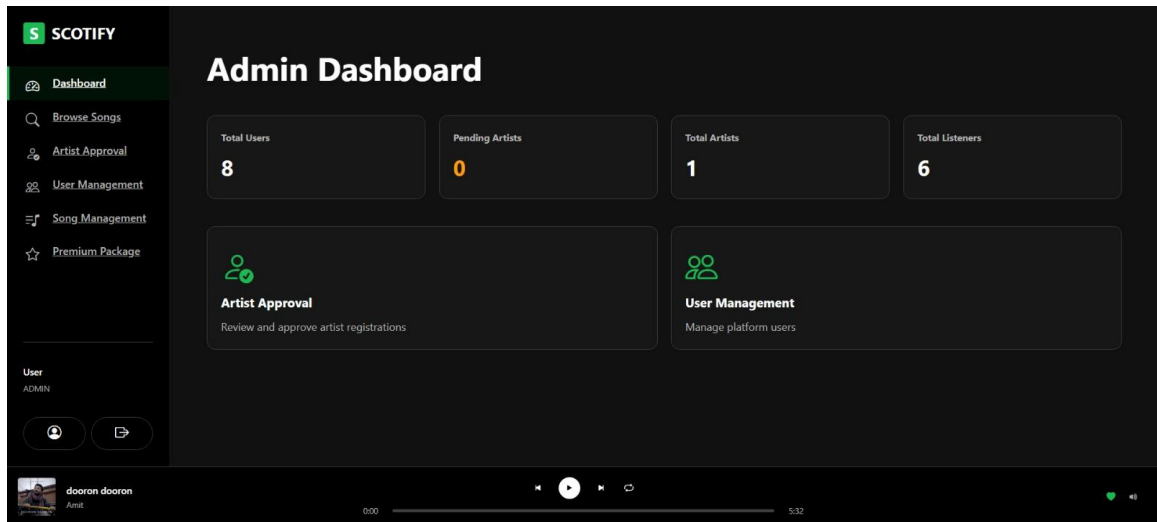
6.7. Listener Premium Page:



6.8. Listener Profile Page:



6.9. Admin Dashboard:



The Admin Dashboard for SCOTIFY features a sidebar with navigation links: Dashboard, Browse Songs, Artist Approval, User Management, Song Management, and Premium Package. The main content area displays four summary cards: Total Users (8), Pending Artists (0), Total Artists (1), and Total Listeners (6). Below these are two action cards: Artist Approval (Review and approve artist registrations) and User Management (Manage platform users). The bottom of the dashboard shows a music player for the song 'dooron dooron' by Amit, with a progress bar at 0:00 and a duration of 5:32.

SCOTIFY

Admin Dashboard

Total Users: 8

Pending Artists: 0

Total Artists: 1

Total Listeners: 6

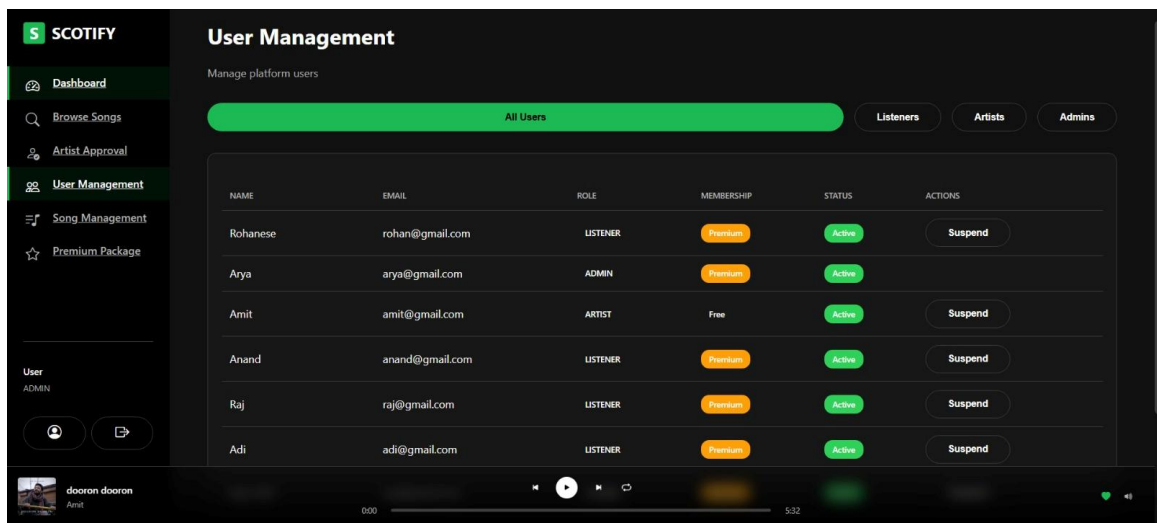
Artist Approval
Review and approve artist registrations

User Management
Manage platform users

User: ADMIN

dooron dooron
Amit
0:00 5:32

6.10. Admin User Management:



The Admin User Management page for SCOTIFY allows managing platform users. It includes a sidebar with navigation links and a main content area with tabs for All Users, Listeners, Artists, and Admins. The All Users tab is active, showing a table of users with columns for Name, Email, Role, Membership, Status, and Actions. The table lists six users: Rohanese, Arya, Amit, Anand, Raj, and Adi. Each user has a 'Suspend' button in the Actions column. The bottom of the page shows a music player for the song 'dooron dooron' by Amit, with a progress bar at 0:00 and a duration of 5:32.

SCOTIFY

User Management
Manage platform users

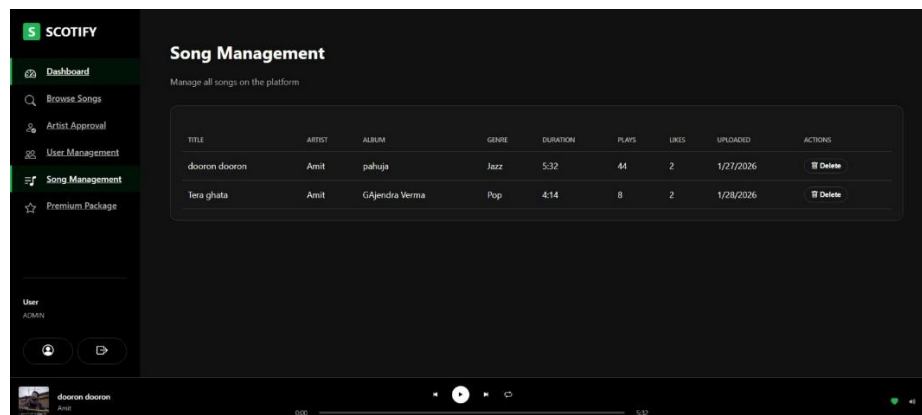
All Users | Listeners | Artists | Admins

NAME	EMAIL	ROLE	MEMBERSHIP	STATUS	ACTIONS
Rohanese	rohan@gmail.com	LISTENER	Premium	Active	Suspend
Arya	arya@gmail.com	ADMIN	Premium	Active	
Amit	amit@gmail.com	ARTIST	Free	Active	Suspend
Anand	anand@gmail.com	LISTENER	Premium	Active	Suspend
Raj	raj@gmail.com	LISTENER	Premium	Active	Suspend
Adi	adi@gmail.com	LISTENER	Premium	Active	Suspend

User: ADMIN

dooron dooron
Amit
0:00 5:32

6.11. Admin Song Management:



The Admin Song Management page for SCOTIFY allows managing all songs on the platform. It includes a sidebar with navigation links and a main content area with a table of songs. The table has columns for Title, Artist, Album, Genre, Duration, Plays, Likes, Uploaded, and Actions. The table lists two songs: 'dooron dooron' by Amit and 'Iera ghata' by Gajendra Verma. Each song has a 'Delete' button in the Actions column. The bottom of the page shows a music player for the song 'dooron dooron' by Amit, with a progress bar at 0:00 and a duration of 5:32.

SCOTIFY

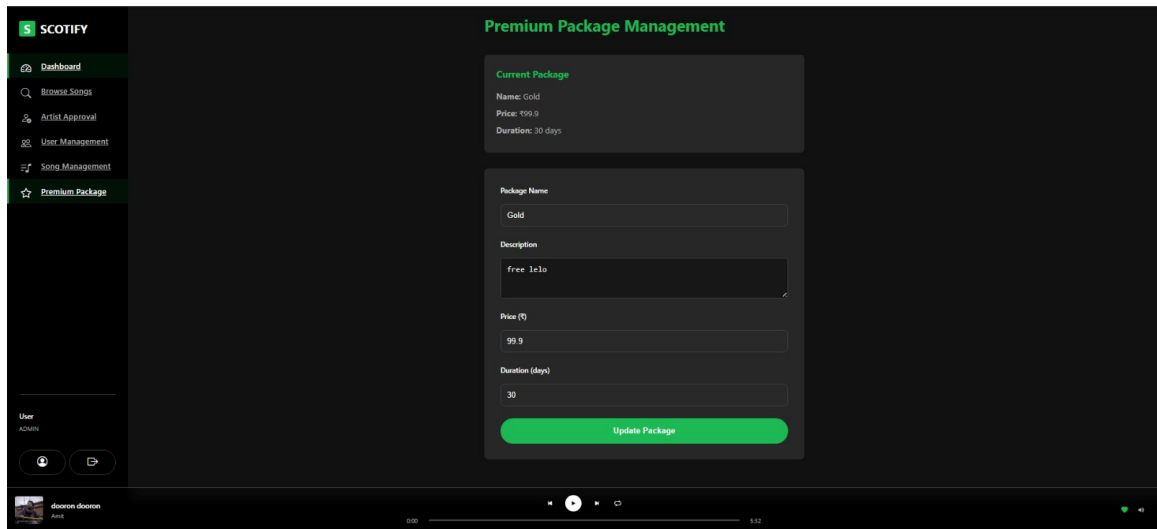
Song Management
Manage all songs on the platform

TITLE	ARTIST	ALBUM	GENRE	DURATION	PLAYS	LIKES	UPLOADED	ACTIONS
dooron dooron	Amit	pahuja	Jazz	5:32	44	2	1/27/2026	Delete
Iera ghata	Amit	Gajendra Verma	Pop	4:14	8	2	1/28/2026	Delete

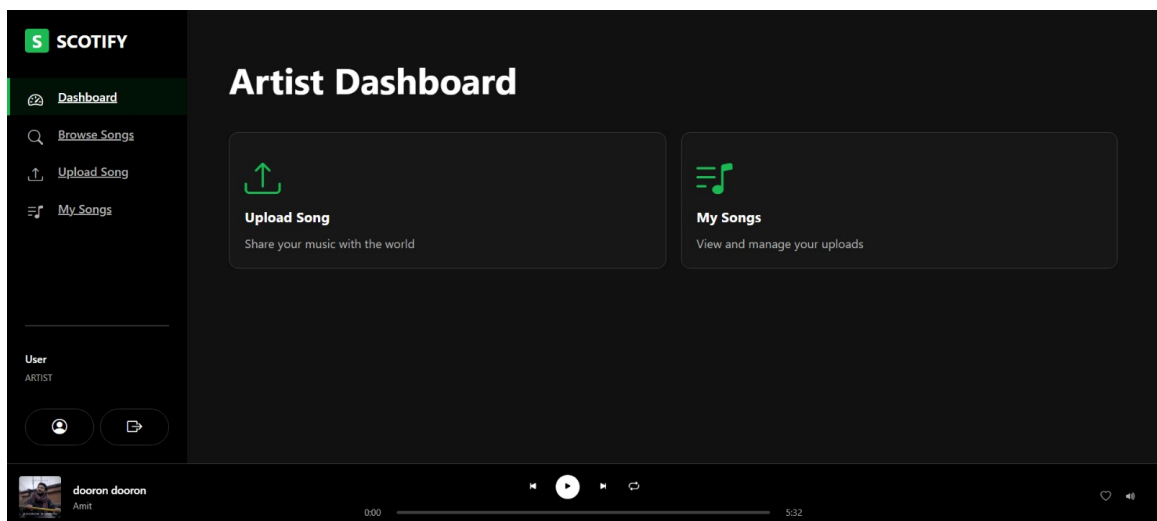
User: ADMIN

dooron dooron
Amit
0:00 5:32

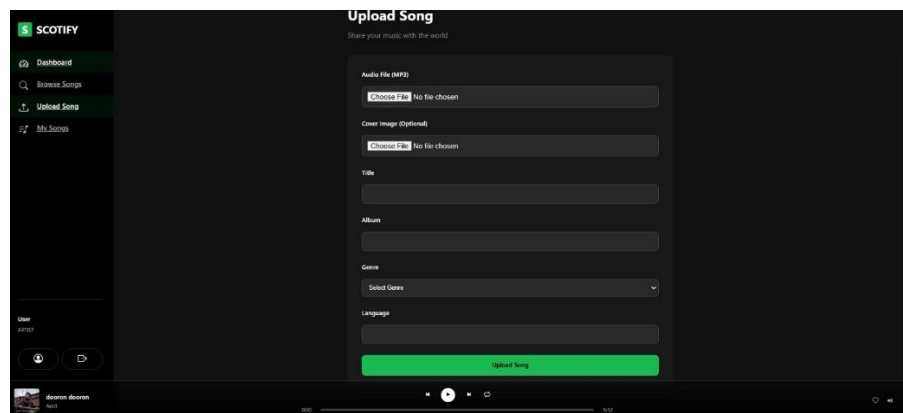
6.12. Admin Premium Package Management:



6.13. Artist Dashboard:



6.14. Artist Song Upload:



6.15. Artist Songs:

The screenshot shows the 'My Songs' page for an artist named 'Amit'. The page has a dark theme. On the left is a sidebar with a menu: Dashboard, Browse Songs, Upload Song, and My Songs (which is highlighted). Below the menu, it says 'User ARTIST'. The main content area is titled 'My Songs' with the subtitle 'Manage your uploaded songs'. It contains a table with two songs:

TITLE	ALBUM	GENRE	DURATION	PLAYS	LIKES	UPLOADED	ACTIONS
dooron dooron	pahuja	Jazz	5:32	45	2	1/27/2026	Edit Delete
Tera ghata	GAjendra Verma	Pop	4:14	8	2	1/28/2026	Edit Delete

At the bottom of the page is a music player for the song 'dooron dooron' by 'Amit', showing a progress bar from 0:00 to 5:32.

6.16. Artist Edit Song:

The screenshot shows a modal form titled 'Edit Song' with a close button (X) in the top right corner. The form contains the following fields and options:

- Genre: A dropdown menu currently showing 'Jazz'.
- Language: A text input field containing 'Hindi'.
- Duration (seconds) *: A text input field containing '332'.
- Replace Audio File (optional): A section with a 'Choose File' button and the text 'No file chosen'.
- Replace Cover Image (optional): A section with a 'Choose File' button and the text 'No file chosen'.
- At the bottom are two buttons: 'Cancel' and 'Update Song'.

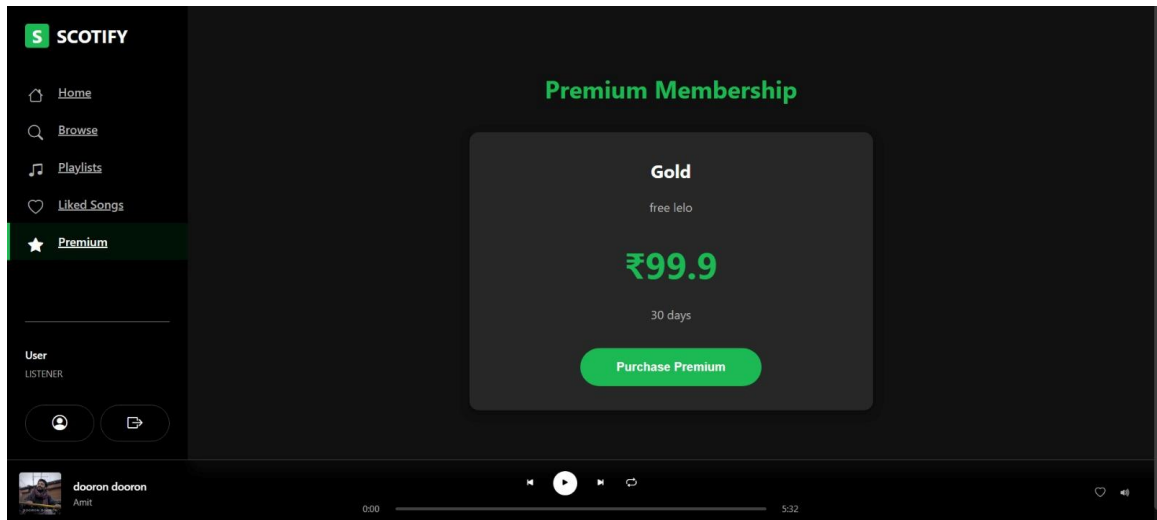
6.17. Admin Artist Approval:

The screenshot shows the 'Artist Approval' page for an admin user named 'ADMIN'. The sidebar menu includes: Dashboard, Browse Songs, Artist Approval (highlighted), User Management, Song Management, and Premium Package. The main content area is titled 'Artist Approval' with the subtitle 'Review and approve artist registrations'. It contains a table with one artist:

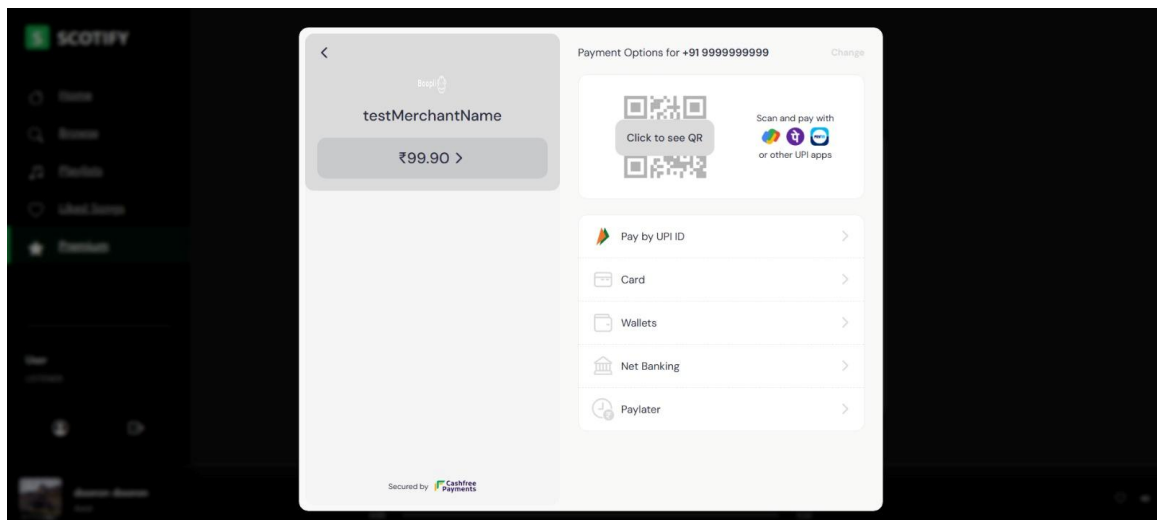
NAME	EMAIL	JOINED DATE	ACTIONS
shreya	shreyai@gmail.com	1/28/2026	Approve Reject

At the bottom of the page is a music player for the song 'dooron dooron' by 'Amit', showing a progress bar from 0:00 to 5:32.

6.18. Listener Buy Premium:



6.19. Cashfree Payment:



7. PROJECT RELATED STATISTICS

The following statistics provide a quantitative overview of the project implementation:

- **Frontend Technology:** React + Vite
- **Backend Technology:** Spring Boot
- **Database Type:** Relational Database (MySQL/PostgreSQL)
- **User Roles Implemented:** 3 (Admin, Artist, Listener)
- **Major Modules:**
 - User Management
 - Content Management
 - Music Streaming
 - Subscription & Payment
- **Media Formats Supported:** MP3 (Audio), JPG/PNG (Images)
- **Payment Gateway Integrated:** Cashfree
- **Security Mechanism:** Role-Based Access Control (RBAC)
- **Total Development Duration:** As per CDAC project timeline
- **Deployment Type:** Web-based Application

These statistics indicate that the project successfully covers all required functional and technical components.

8. CONCLUSION

The **SCOTIFY – Online Music Store** project successfully delivers a secure, scalable, and user-friendly web-based music distribution platform. By implementing a modern full-stack architecture using React + Vite and Spring Boot, the system effectively bridges the gap between music creators and listeners.

The platform supports secure role-based access control, efficient content management for artists, seamless streaming for listeners, and reliable monetization through premium subscriptions powered by Cashfree payment integration. The project fulfills all defined objectives and demonstrates practical application of web technologies, database management, and security principles.

Overall, SCOTIFY serves as a comprehensive academic solution that reflects real-world online music platforms and provides a strong foundation for future enhancements such as recommendation systems, analytics, and mobile application support.

9. REFERENCES

1. <https://open.spotify.com/>
2. <https://music.apple.com/us/new>
3. <https://www.jiosaavn.com/>
4. <https://music.youtube.com/>
5. <https://soundcloud.com/>